

Project Report - Deep Neural Network Compression Pipeline

Abstract

This document describes a three-stage compression pipeline for a VGG-style convolutional neural network trained on CIFAR-10. The project follows the ideas presented in the research paper *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding* by Han, Mao, and Dally, available at <https://arxiv.org/pdf/1510.00149>. The goal of the implementation is to reduce model size and memory usage while preserving classification accuracy, making the network more suitable for resource-constrained deployment.

1 Introduction

This section details the model architecture, optimization mathematics, and the iterative pruning strategy used to compress a VGG-style neural network for efficient deployment on the CIFAR-10 dataset.

Deep neural networks typically achieve high accuracy, but they are expensive to store and deploy because of their large parameter counts and high memory bandwidth requirements. The compression approach used in this project is based on the deep compression framework introduced in the paper by Han et al. The original method combines pruning, trained quantization, and Huffman coding to reduce both storage cost and inference overhead.

In this project, the pipeline is implemented in three stages:

1. **Pruning**: remove low-magnitude weights and enforce sparsity.
2. **Quantization**: replace remaining weights with shared centroid values.
3. **Compression**: serialize the quantized sparse model into a compact file.

The model is trained and evaluated on **CIFAR-10**, a standard image classification dataset containing 32×32 RGB images across 10 classes.

2 Model Architecture, Pruning, and Optimization

The network used in this project is a lightweight VGG-style CNN consisting of convolutional layers for feature extraction, pooling layers for spatial downsampling, and fully connected layers for classification. The architecture is intentionally kept simple to clearly observe the effects of compression.

2.1 Loss Function and Optimization

The model is trained using the **Cross-Entropy Loss**:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Optimization is performed using the **Adam Optimizer**, which adapts learning rates using first and second moment estimates:

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

During training, the network learns a dense parameter set $W = \{w_i\}_{i=1}^N$, which serves as the baseline for subsequent compression.

2.2 Iterative Magnitude Pruning

To achieve compression, magnitude-based pruning is applied with **PRUNE_RATE = 0.20**. At each cycle, a threshold τ is computed:

$$\tau = \text{Percentile}(|W|, 20\%)$$

Weights are pruned according to:

$$\tilde{w}_i = \begin{cases} w_i & \text{if } |w_i| > \tau \\ 0 & \text{otherwise} \end{cases}$$

A binary mask M is defined:

$$M_i = \begin{cases} 1 & \text{if } |w_i| > \tau \\ 0 & \text{otherwise} \end{cases}$$

Thus, pruning is expressed as:

$$\tilde{W} = W \odot M$$

After 10 pruning cycles, the number of active parameters reduces from approximately 4.27×10^6 to 4.59×10^5 , resulting in a sparsity of **89.26%**. Masks are enforced during training to ensure pruned weights remain zero.

2.3 Implementation using PyTorch

The model and training pipeline are implemented using the **PyTorch** deep learning framework, which provides high-level abstractions for tensor operations, automatic differentiation, and optimization.

The neural network is defined as a subclass of `torch.nn.Module`, where layers such as convolution, activation, pooling, and fully connected layers are composed sequentially. The forward propagation is implemented through the `forward()` function, which defines how input data flows through the network.

Training is performed using PyTorch’s built-in components:

- The loss function is implemented using `torch.nn.CrossEntropyLoss()`, which internally computes the softmax activation followed by the negative log-likelihood loss.
- Optimization is handled using `torch.optim.Adam`, which updates model parameters based on gradients computed via backpropagation.

Automatic differentiation is managed by PyTorch’s `autograd` engine. During training, gradients are computed using:

- `loss.backward()` to compute gradients
- `optimizer.step()` to update parameters

Additionally, pruning is integrated into the training loop by applying binary masks to gradients. This ensures that pruned weights remain zero throughout training. All tensor operations, including masking, pruning, and quantization, are performed using PyTorch tensors, allowing efficient computation on both CPU and GPU.

The use of PyTorch simplifies the implementation of the compression pipeline while ensuring computational efficiency and flexibility for experimentation.

3 Training and Rewinding Strategy

Following each pruning cycle, the remaining weights are rewound to their initial values:

$$W_{\text{rewound}} = W_{\text{init}} \odot M$$

This approach, inspired by the Lottery Ticket Hypothesis, allows sparse subnetworks to be retrained effectively. The training loop alternates between training, pruning, mask application, and rewinding, ensuring stable optimization despite increasing sparsity.

4 Trained Quantization

After pruning, only the active weights $W_a = \{w_i \mid M_i = 1\}$ are quantized using **K-Means clustering**. The objective is to minimize within-cluster variance:

$$\min_C \sum_{k=1}^K \sum_{w \in C_k} \|w - \mu_k\|^2$$

Each weight is approximated by its cluster centroid:

$$w_i \approx \mu_{z_i}$$

where $z_i \in \{1, \dots, K\}$ is the cluster index. The number of clusters is determined as:

$$K = 2^b$$

This replaces full-precision weights with shared centroid values, reducing storage requirements.

4.1 Memory Reduction Analysis

Original model:

$$\text{Memory}_{\text{original}} = 32 \times N \text{ bits}$$

After pruning:

$$\text{Memory}_{\text{pruned}} \approx 32 \times N_{\text{active}}$$

After quantization:

$$\text{Memory}_{\text{quantized}} = N_{\text{active}} \cdot \log_2 K + 32K$$

For example, with $K = 8$:

$$\log_2 K = 3 \text{ bits per weight}$$

Empirically, this results in:

- Original size: ~ 16.31 MB
- Compressed size: ~ 1.59 MB
- Compression ratio: $\sim 10.28\times$

Thus, pruning reduces the number of parameters while quantization reduces the number of bits per parameter, leading to substantial overall compression.

5 Compression and Serialization

The final stage stores the quantized sparse weights in a compact file using compressed NumPy serialization. This step acts as the practical storage layer of the pipeline.

Although the original paper uses explicit Huffman coding, this implementation relies on compressed archive storage to achieve a similar purpose at the serialization stage. The compressed state contains:

- quantized weight arrays,
- bias tensors,
- layer-wise parameter data.

The resulting file is much smaller than a dense floating-point model checkpoint. This makes the model easier to distribute and deploy.

6 Deployment Workflow

The compressed model can be loaded back into a fresh instance of the same architecture. During deployment, the saved arrays are read from disk and copied into the model parameters.

The deployment process is:

1. load the compressed file,
2. create a new model instance,
3. inject the stored weights,
4. switch the model to evaluation mode,
5. run inference on a test input.

This confirms that the compressed model remains functional after the full compression pipeline.

7 Project Workflow

The complete pipeline is organized as follows:

1. **Dataset preparation:** CIFAR-10 is loaded and normalized.
2. **Baseline training:** the VGG-style CNN is trained on the dataset.
3. **Iterative pruning:** low-magnitude weights are removed cycle by cycle.
4. **Fine-tuning:** the sparse model is retrained to recover accuracy.
5. **K-Means quantization:** surviving weights are clustered into shared centroids.
6. **Compression:** the quantized parameters are stored in a compact archive using Huffman Coding.
7. **Deployment:** the compressed model is reloaded and evaluated.

8 Code Structure and Implementation Details

The project is organized into multiple modules, each responsible for a specific stage of the compression pipeline. This modular design improves clarity, reusability, and ease of debugging.

8.1 Main Execution (`main.py`)

The `main.py` file serves as the entry point of the project and orchestrates the entire pipeline. It performs the following steps:

- Initializes the model and loads the CIFAR-10 dataset
- Executes iterative pruning cycles along with retraining
- Applies quantization to the pruned model
- Saves the compressed model to disk
- Computes compression statistics

8.2 Configuration (`config.py`)

This file stores all hyperparameters and settings used throughout the project, including:

- Number of pruning cycles
- Pruning rate
- Number of quantization bits
- Batch size and learning rate
- Device configuration (CPU/GPU)

8.3 Model Definition (`models/model.py`)

This file defines the VGG-style convolutional neural network architecture. It includes:

- Convolutional layers for feature extraction
- Pooling layers for spatial downsampling
- Fully connected layers for classification
- A forward pass defining data flow through the network

8.4 Data Loading (`data/data_loader.py`)

Responsible for preparing the dataset:

- Loads CIFAR-10 using PyTorch utilities
- Applies normalization and augmentation transforms
- Creates data loaders for training and testing

8.5 Training Utilities (`utils/train.py`)

Contains core training and evaluation logic:

- `train_model()`: Performs forward pass, loss computation, backpropagation, and parameter updates
- `test_model()`: Evaluates model accuracy on the test dataset

Additionally, gradient masking is applied to ensure pruned weights remain zero during training.

8.6 Metrics (`utils/metrics.py`)

Provides helper functions to evaluate compression:

- Computes sparsity percentage
- Tracks number of active versus total parameters

8.7 Pruning Module (`compression/prune.py`)

Implements magnitude-based pruning:

- Computes thresholds based on weight magnitude
- Updates binary masks for each layer
- Removes low-importance weights

8.8 Weight Rewinding (`compression/rewind.py`)

Handles the rewinding step after pruning:

- Restores remaining weights to their initial values
- Applies current pruning mask

8.9 Quantization Module (`compression/quantize.py`)

Implements K-Means based quantization:

- `quantize_tensor()`: Clusters non-zero weights into centroids
- `quantize_model()`: Applies quantization layer-wise across the model

8.10 Compression Module (`compression/huffman.py`)

Handles final model storage:

- Saves model using `np.savez_compressed`
- Computes compression ratio and memory usage

8.11 Deployment Module (`deployment/load_model.py`)

Responsible for loading and testing the compressed model:

- Loads compressed weights from disk
- Reconstructs model parameters
- Performs inference on sample inputs

8.12 Analysis Module (`analysis/kmeans_elbow.py`)

Used for selecting optimal number of clusters:

- Runs K-Means for multiple values of K
- Computes within-cluster variance
- Identifies optimal cluster count using the elbow method

Overall, the codebase follows a structured pipeline where each module corresponds to a specific stage in the compression process, ensuring clarity and maintainability.

9 Conclusion and Results

This project successfully demonstrates a complete deep neural network compression pipeline inspired by the work of Han et al. The implementation integrates pruning, quantization, and compression into a unified framework applied to a VGG-style CNN trained on CIFAR-10.

The iterative pruning process progressively reduced the number of active parameters from approximately 4.27 million to 459,005, achieving a final sparsity of **89.26%**. Despite this significant reduction, the model maintained a stable test accuracy of approximately **72.6%**, indicating that most redundant parameters were effectively removed without severely degrading performance.

Following pruning, K-Means quantization was applied to the remaining weights. An elbow-method analysis determined an optimal clustering value of $K = 6$, which was approximated using a power-of-two configuration for compatibility. This step further reduced the precision of weights while preserving representational capacity.

The final compression stage produced a compact model file of size **1.59 MB**, compared to the original theoretical size of **16.31 MB**. This corresponds to a compression ratio of approximately **10.28×**, exceeding the target compression ratio of $9\times$. The compressed model was successfully reloaded and validated through inference, confirming that the deployment pipeline remains functional.

The implementation demonstrates the practical feasibility of deploying deep neural networks in memory-constrained environments, aligning closely with the principles outlined in the original deep compression research.

Type	Value
Best observed (mid-training)	$\sim 73.82\%$
Final compressed model	$\sim 72.60\%$
Estimated drop	$\sim 1.2\%$

Reference

Han, S., Mao, H., & Dally, W. J. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. Available at: <https://arxiv.org/pdf/1510.00149>